Gen AI Documentation

Date Used: March 12, 2026

Tool disclosure: AI Agent on Visual Studio Code (CoPilot)

Purpose of Use: To explore AI assisted coding to build a simple interactive game. The goal is to explore basic physics and sound effects. I wanted to create a small game where the ball's movement and player actions would trigger different visual and audio responses to make the gaming experience more engaging.

Summary of Interaction: I used Copilot in Visual Studio Code to help write the code for the game. I prompted AI to help create the ball's movement and also the platform controls and star spawning/collecting system. I also added sound effects so that when the ball hits a wall it changes colours and plays a random musical note. Stars are also collectable and a short sound plays when one is collected.

Human Decision Points:

- Decided to make a simple bouncing ball and moving platform game mechanic
- Chose to have collectible stars as the main objective of the game
- Decided that each wall would be a different colour and that the ball would change colours when it hits them
- Added sound effects for wall bounces, star collection, and game over

Scope of GenAI Use: GenAI was used to write all of the code and comments. GenAI was not used to write any of the project descriptions, reflections, or Read Me content.

Limitations or Misfires:

- The moving platform didn't initially affect the direction of the ball very much, making it hard to aim to reach the stars
- Some of the AI generated code made the sound not play properly or at the correct time
- Stars were sometimes spawned too close to each other or in unreachable spots
- Trial and adjustments were needed to make the game feel more balanced

Summary of Process (Human + Tool)

- Prompting and iteration
- Playtesting
- Manual adjustments

Decision Points & Trade-offs

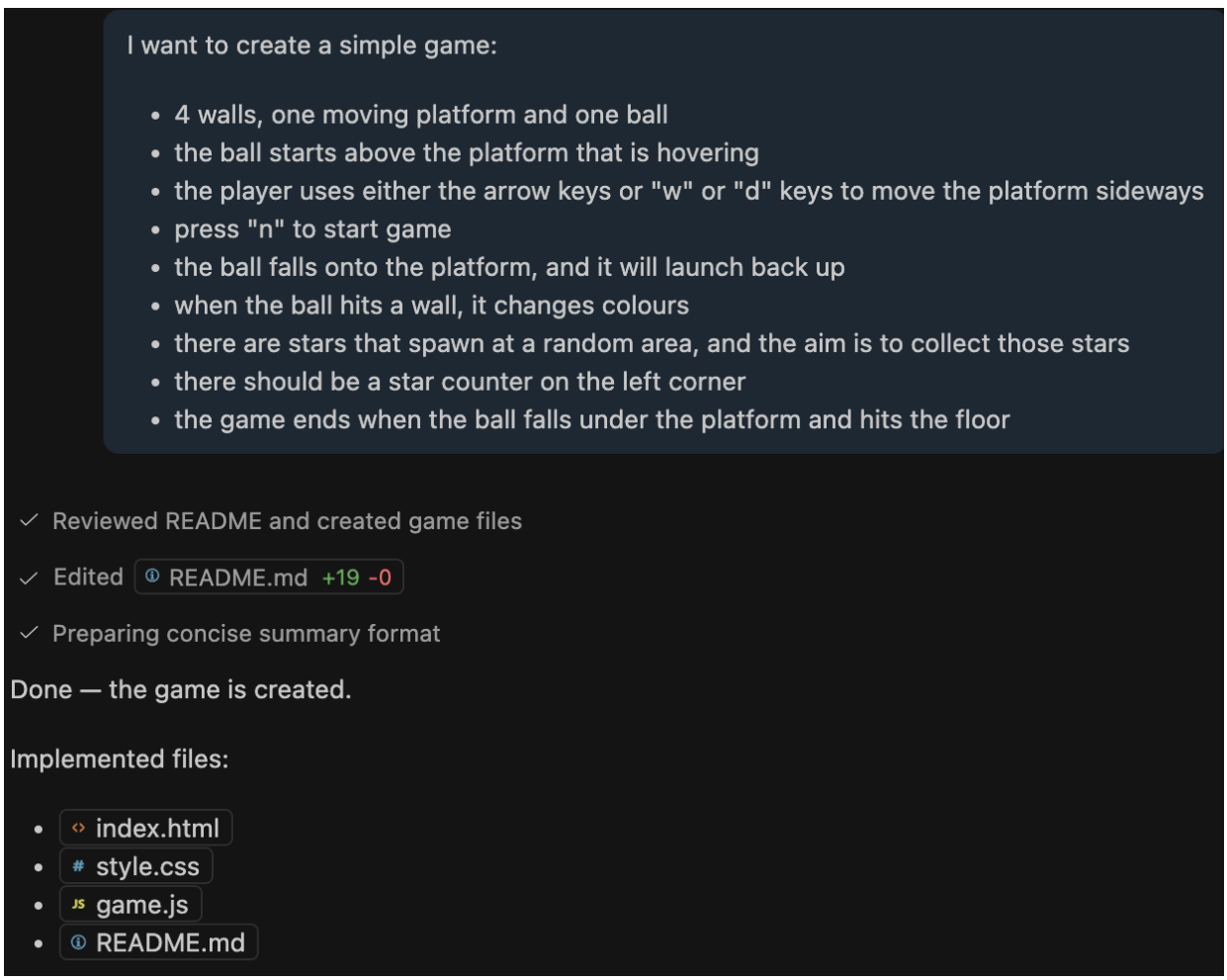
- Adjusted the physics of the platform to have a stronger influence on the ball's bounce direction so that the player could control where the ball goes better
- Made each wall a different colour, and had the ball change to its colour when it hits the wall
- Added random musical notes for every wall bounce instead of a single sound to make the game more interesting and dynamic
- Kept the number of game elements small for easier debugging

Verification & Judgement

- I ran the game in the browser and playtested in multiple times
- I checked that the sound effects triggered correctly during any ball bounces, star collection, and game over
- I adjusted the values and mechanics until the game felt responsive and playable

---

APPENDIX



What's included:

- 4 walls, 1 movable platform, 1 ball
- Ball starts above hovering platform
- Movement with Left/Right arrows and `W`/`D` (`W` maps to left, `D` to right)
- Press `N` to start/restart
- Ball falls, bounces off platform, and launches upward
- Ball changes color on wall hit
- Random star spawning and collection by ball
- Star counter in top-left
- Game over when ball drops below platform and hits the floor

Open index.html in a browser to play.

Restore Checkpoint

the ball should go against gravity, when it hits a surface, it travels in the direction it is hit in until it bounces off another surface

Click to Edit

✓ Inspected game.js and implemented anti-gravity bounce

✓ Edited game.js +93 -58

✓ Updated error check in game.js

Updated. The ball now uses anti-gravity style movement in game.js :

- No downward acceleration from gravity.
- Ball keeps a constant speed and travels in a straight direction until collision.

- Wall/top collisions reflect direction and still change color.
- Platform collision now sets outgoing direction based on where the ball hits (and slight platform movement influence), then continues straight.
- Game-over behavior on floor hit is unchanged.

If wanted, I can also make `W` = up and `D` = right instead of using `W` as left.

---

is there a better mechanic that would make the ball reach the stars easier? right now the platform has not much impact at where the ball is directed.

✓ Inspected game mechanics and designed bounce enhancement

✓ Edited game.js +28 -4

✓ Checked game.js file

Yes—good call. I added a better mechanic and implemented it in game.js:

- Stronger paddle influence:
  - Bounce angle now depends more on where the ball hits the platform (curved response for better control).
  - Platform movement momentum (platform.vx) now pushes the outgoing direction more clearly.
- Star assist on paddle bounce:
  - After each platform hit, the ball gets a light steering nudge toward the current star (only if the star is above), so stars are easier to reach without feeling fully auto-aimed.

This should feel much more controllable while still keeping bounce gameplay.

have each wall be a different colour (the colour that the ball will change into)

✓ Refactoring wall colors and ball bounce behavior

✓ Edited game.js +17 -19

✓ Checked game.js file

Done. Each wall now has its own fixed color, and the ball changes to that exact wall color on collision in game.js.

Also updated:

- Top, left, right, and bottom walls are visibly different colors.
- If the ball hits the floor (game over), it changes to the bottom wall's color too.

GPT-5.3-Codex • 0.9x

---

make the platform white

✓ Modifying platform fill style

✓ Edited game.js +1 -1

Done. The platform is now white in game.js.

when the ball hits a wall, a random musical note is played

✓ Planned audio context note and implemented audio

✓ Edited game.js +45 -0

✓ Checked game.js file

Done. I added random musical notes on wall collisions in game.js:

- Added a small Web Audio synth (AudioContext + oscillator).
- Each wall hit now plays a random note from a musical scale.
- Floor hit (game over) also plays a note, since it's the bottom wall.

If you want, I can map specific notes per wall color (e.g., top = C, left = E, right = G, bottom = B) instead of fully random.

when the game fails, play "fail.mp3"

✓ Adding fail sound on gameover

✓ Edited game.js +11 -0

✓ Checked game.js file

Done. On game over, the game now plays `fail.mp3` in game.js.

Make sure fail.mp3 is in the project root (same folder as index.html).

when a star is collect, play the "star.mp3" audio from 0:00 to 0:02

✓ Inspected game.js and implemented sound playback

✓ Edited game.js +18 -0

✓ Checked game.js file

Done. I added star collection audio in game.js:

- Uses star.mp3
- Plays on each star pickup
- Stops after exactly 2 seconds (0:00 to 0:02)

Make sure star.mp3 is in the project root (same folder as index.html).